

```
!pip install transformers datasets accelerate -q

import torch, math
from transformers import (GPT2LMHeadModel, GPT2Tokenizer, Trainer,
                          TrainingArguments, DataCollatorForLanguageModeling, set_seed)
from datasets import Dataset

set_seed(42)
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
model = GPT2LMHeadModel.from_pretrained('gpt2')
tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = tokenizer.eos_token_id

def generate_text(model, tokenizer, prompt, max_length=100):
    model.eval()
    inputs = tokenizer.encode(prompt, return_tensors='pt').to(model.device)
    with torch.no_grad():
        out = model.generate(inputs, max_length=max_length, temperature=0.8,
                             top_k=50, do_sample=True, pad_token_id=tokenizer.eos_token_id)
    return tokenizer.decode(out[0], skip_special_tokens=True)

review_prompts = [
    'This product is',
    'I bought this phone and',
    'The quality of this item',
]

print('=== BASELINE REVIEWS (Before Fine-Tuning) ===')
baseline = {}
for p in review_prompts:
    baseline[p] = generate_text(model, tokenizer, p)
    print(f'Prompt: {p}\nOutput: {baseline[p]}\n')
```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), s
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fa
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN
tokenizer_config.json: 100% 26.0/26.0 [00:00<00:00, 2.15kB/s]

vocab.json: 1.04M/? [00:00<00:00, 2.46MB/s]

merges.txt: 456k/? [00:00<00:00, 955kB/s]

tokenizer.json: 1.36M/? [00:00<00:00, 2.35MB/s]

config.json: 100% 665/665 [00:00<00:00, 68.6kB/s]

model.safetensors: 100% 548M/548M [00:03<00:00, 405MB/s]

Loading weights: 100% 148/148 [00:00<00:00, 453.57it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key | Status | |
-----+-----+
h.{0...11}.attn.bias | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
generation_config.json: 100% 124/124 [00:00<00:00, 7.34kB/s]

The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence,
=== BASELINE REVIEWS (Before Fine-Tuning) ===
Prompt: This product is
Output: This product is made from high quality, lightweight stainless steel. If you are looking for something a little mor

Laser Pouch

Not all of our laser printers are created equal. We have a laser printer that comes with all of our printer parts. These p

Prompt: I bought this phone and
Output: I bought this phone and I have not used it on a lot of people. I have also not used it on any other people.

The screen was amazing and the sound was amazing. It was not loud. I would never use it on a tv, laptop, smartphone or oth

The battery life is good. The phone works great but it has so many problems.

I have been using phones that have the Snapdragon 616 processor with the 8

Prompt: The quality of this item
Output: The quality of this item in the item description (and if the item is already in stock) will determine how many tim

This item is currently out of stock and may be out of stock based on the availability of your specific request.

Please contact us before placing your order to verify availability.

Your order is now up for pre-order.

```

```

corpus = [
    'this phone has an amazing battery life and the camera quality is outstanding for the price.',
    'i bought this laptop for college and it handles all my assignments and coding projects perfectly.',
    'the sound quality of these headphones is incredible with deep bass and clear vocals.',
    'this smartwatch tracks my steps accurately and the heart rate monitor is very reliable.',
    'great wireless earbuds with noise cancellation that blocks out all background sound.',
    'the keyboard feels very comfortable for long typing sessions and the backlight is a nice touch.',
    'this portable charger saved me during travel and it charges my phone three times on a single charge.',
    'the tablet screen is bright and colorful which makes watching movies a great experience.',
    'i love this fitness tracker because it motivates me to reach my daily exercise goals.',
    'this bluetooth speaker is compact but delivers surprisingly loud and clear audio.',
    'the delivery was fast and the product was packed securely with no damage at all.',
    'excellent value for money and the build quality feels premium despite the affordable price.',
    'the customer service team was very helpful when i had questions about the product features.',
    'this camera takes stunning photos in low light and the video recording quality is very smooth.',
    'i have been using this product for three months and it still works perfectly like day one.',
    'the design is sleek and modern and it looks great on my desk next to my other gadgets.',
    'easy to set up right out of the box and the instructions were clear and simple to follow.',
    'highly recommend this product to anyone looking for quality and reliability at a fair price.',
    'the software updates keep adding new features which makes this purchase even more worthwhile.',
    'best purchase i made this year and i would definitely buy from this brand again.',
]

dataset = Dataset.from_dict({'text': corpus})
tokenized = dataset.map(lambda x: tokenizer(x['text'], truncation=True,
                                         max_length=128, padding='max_length'), batched=True, remove_columns=['text'])

```

Map: 100%

20/20 [00:00<00:00, 275.95 examples/s]

```
training_args = TrainingArguments(
    output_dir='./gpt2-reviews', num_train_epochs=15,
    per_device_train_batch_size=4, learning_rate=5e-5,
    weight_decay=0.01, warmup_steps=50, eval_strategy='epoch',
    logging_steps=10, save_strategy='no',
    fp16=torch.cuda.is_available(),
)

trainer = Trainer(model=model, args=training_args,
                  train_dataset=split['train'], eval_dataset=split['test'],
                  data_collator=data_collator)

trainer.train()
```

`loss\_type=None` was set in the config but it is unrecognized. Using the default loss: `ForCausalMLoss`. [75/75 00:12, Epoch 15/15]

Epoch	Training Loss	Validation Loss
1	No log	3.348063
2	4.057265	3.247350
3	4.057265	3.106139
4	3.319935	2.981021
5	3.319935	2.881039
6	2.706133	2.803390
7	2.706133	2.756230
8	1.918991	2.740822
9	1.918991	2.765902
10	1.234805	2.867215
11	1.234805	3.047834
12	0.617781	3.198001
13	0.617781	3.197028
14	0.437045	3.189241
15	0.437045	3.208345

TrainOutput(global_step=75, training_loss=1.9250287294387818, metrics={'train_runtime': 14.4057, 'train_samples_per_second': 17.701, 'train_steps_per_second': 5.206, 'total_flos': 16657367040000.0, 'train_loss':

```
eval_res = trainer.evaluate()
print(f'Perplexity: {math.exp(eval_res["eval_loss"]):.2f}')
```